

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №1
по дисциплине «Методы тестирования
программного обеспечения»
«Инспекция кода»

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Изучение методов ручного тестирования	5
2.1 Инспекция кода	5
2.2 Сквозные просмотры	6
2.3 Проверка за столом	7
2.4 Рецензирование	7
3 Описание тестируемой программы	9
3.1 Спецификация	9
3.2 Исходный код программы	10
4 Описание проведенной инспекции кода	15
4.1 Протокол заседания	15
4.2 Полученные рекомендации	20
4.3 Исправление кода программы	21
Заключение	28
Список использованной литературы	29

Введение

В данной лабораторной работе представлено описание использования одного из методов ручного тестирования - инспекция кода.

Тестирование программного обеспечения - это процесс проверки ПО на соответствие спецификации и с целью выявления ошибок.

Ручное тестирование - это процесс тестирования, не предполагающий использования компьютера и осуществляемый непосредственно человеком. Оно зарекомендовало себя достаточно эффективным методом обнаружения ошибок, и его разновидности следует задействовать в любом программном проекте.

Методы ручного тестирования должны применяться к написанному исходному коду до того, как начнется его тестирование на компьютере. Можно также самостоятельно создавать аналогичные методики и применять их на более ранних стадиях процесса программирования (например, в конце каждого этапа разработки проекта).

1 Постановка задачи

В ходе выполнения данной лабораторной работы требуется провести инспекцию кода, выступая в роли разработчика программы.

Для выполнения данной задачи необходимо:

- изучить методы ручного тестирования;
- провести инспекцию кода;
- проанализировать результаты тестирования;
- исправить программу в соответствии с рекомендациями специалиста по тестированию.

2 Изучение методов ручного тестирования

Основными методами ручного тестирования являются инспекция кода, сквозной просмотр и тестирование удобства использования. Эти методы могут применяться практически на любой стадии разработки программного обеспечения, причем как к отдельным готовым модулям или блокам, так и к приложению в целом.

2.1 Инспекция кода

Инспекция кода — это набор процедур и методик обнаружения ошибок путем анализа (чтения) кода группой специалистов. В большинстве случаев, говоря об инспекции кода, основное внимание уделяют процедурным вопросам, формам, подлежащим заполнению, и т.п.

Группа инспектирования кода

Обычно в состав группы входят четыре человека, один из которых играет роль координатора. Он должен быть квалифицированным программистом, но не автором тестируемой программы, детальное знание которой от него не требуется. В его обязанности входит следующее:

- раздача материалов участникам заседания инспекционной группы и составление плана его проведения;
- проведение заседания;
- запись всех обнаруженных ошибок;
- последующая проверка того, что обнаруженные ошибки устранены.

Вторым участником группы является программист, а остальными — проектировщик программы (если это не сам программист) и специалист по тестированию. Последний должен не только хорошо ориентироваться в методах тестирования ПО, но и знать наиболее распространенные типы ошибок.

Порядок проведения заседаний по инспектированию кода

За несколько дней до заседания координатор раздает листинг программы и спецификацию проекта всем участникам группы. Это делается для того, чтобы к моменту проведения заседания участники успели ознакомиться с необходимыми материалами. Инспекционное заседание проводится следующим образом.

1. Программист рассказывает присутствующим о логике работы программы, подробно останавливаясь на каждой инструкции. В это

время другие участники заседания должны задавать ему вопросы, которые поспособствуют выявлению возможных ошибок.

2. Участники заседания анализируют код программы, сверяясь с составленными на основе имеющегося опыта контрольными списками наиболее распространенных ошибок.

Координатор обязан направлять дискуссию в конструктивное русло и следить за тем, чтобы участники концентрировали свое внимание не на устранении ошибок, а на их обнаружении.

По окончании заседания программисту передается список найденных ошибок. Если этот список достаточно длинный или устранение ошибок требует внесения существенных изменений в программу, координатор может принять решение о повторной инспекции кода после того, как ошибки будут исправлены. Список анализируется и после разбиения ошибок на категории дополняет имеющийся контрольный список, что позволяет повысить эффективность будущих инспекций кода.

Время и место проведения инспекции должны быть спланированы так, чтобы никакие внешние факторы не могли мешать работе заседания. Оптимальная длительность таких заседаний составляет от полутора до двух часов.

2.2 Сквозные просмотры

Как и инспекция, сквозной просмотр кода представляет собой совокупность процедур и методов обнаружения ошибок в коде путем его просмотра участниками группы тестирования. Такой просмотр имеет много общего с процессом инспектирования, но при этом используются несколько иные процедуры и методы.

Сквозной просмотр проводится в форме непрерывного заседания, длящегося не более двух часов. Число участников группы, выполняющей сквозной просмотр, составляет 3-5 человек. Один из них выполняет функции, аналогичные функциям координатора в группе инспектирования. Еще один человек играет роль секретаря и записывает все найденные ошибки, а третий участник — тестировщик. Среди членов группы также должен быть программист, написавший программу.

Процедура сквозного просмотра начинается точно так же, как и процедура инспекции: за несколько дней до заседания необходимые материалы раздают участникам, чтобы они могли заблаговременно ознакомиться с проверяемым программным обеспечением. Однако работа самого заседания организуется иначе. Вместо простого чтения кода программы

или ее проверки с помощью контрольного списка ошибок участники заседания «играют в компьютер». Тот, кто был назначен тестировщиком, приходит на совещание с предварительно записанными на листах бумаги тестами — представительными наборами входных (и ожидаемых выходных) данных для программы или модуля. Во время заседания участники «прогоняют в уме» каждый тест, т.е. подвергают тестовые данные обработке вручную в соответствии с логикой программы.

Тесты служат средством «разогрева» участников и основой для постановки вопросов программисту, касающихся логики проектирования и принятых допущений. В большинстве сквозных просмотров при выполнении самих тестов находят меньше ошибок, чем при опросе программиста.

2.3 Проверка за столом

Метод ручного тестирования «Проверка за столом» может рассматриваться как инспекция или сквозной просмотр кода, выполняемые одним человеком, который вычитывает код программы, проверяет его, руководствуясь контрольным списком ошибок, и (или) прогоняет через логику программы тестовые данные.

Для большинства людей проверка за столом является относительно непродуктивной. Это объясняется прежде всего тем, что такая проверка представляет собой полностью неупорядоченный процесс. Вторая причина заключается в том, что проверка за столом вступает в противоречие со вторым принципом тестирования, согласно которому тестирование программистом собственных программ обычно оказывается неэффективным. Следовательно, оптимальный вариант состоит в том, чтобы такую проверку выполнял человек, не являющийся автором программы (например, два программиста могут обмениваться программами для взаимной проверки, а не проверять собственные программы), но даже в этом случае такая проверка менее эффективна, чем сквозные просмотры или инспекции. В основном именно по этой причине лучше, чтобы сквозные просмотры или инспекции осуществлялись в группе.

2.4 Рецензирование

Рецензирование - это процедура анонимной оценки общих характеристик качества, обслуживаемости, расширяемости, удобства использования и ясности программного обеспечения.

Цель данного метода - предоставить программисту возможность получить стороннюю оценку результатов своего труда.

Общее руководство процессом осуществляет администратор, выбираемый из числа программистов. В свою очередь, администратор отбирает в группу рецензентов от 6 до 20 участников. Предполагается, что все участники специализируются в одной области. Каждый из участников предоставляет для рецензирования две своих программы, одну из которых он считает наилучшей, а вторую — наихудшей по качеству.

После сбора всех программ их распределяют случайным образом между участниками. Каждому участнику дают для рецензирования четыре программы: две «лучших» и две «худших», но рецензенту не сообщают, какой именно является каждая из них. Рецензента просят предоставить свои замечания к программе и дать рекомендации по ее улучшению.

После просмотра всех программ каждому участнику передают анкеты с оценками двух его программ. Кроме того, участники получают статистическую сводку, отражающую общие и детализированные данные о рейтинге их собственных программ среди всего набора, а также анализ того, насколько оценки, данные участником чужим программам, близки к оценкам тех же программ со стороны других рецензентов. Цель всего этого процесса — предоставить программистам возможность самим оценить уровень своей квалификации.

3 Описание тестируемой программы

Программа должна реализовывать интерактивный калькулятор в вещественных числах, поддерживающий следующие операции:

- Сложение, умножение, вычитание, деление, степень.
- Скобки.
- Унарный минус.
- Операцию взятия предыдущего результата `_`.

Также калькулятор должен:

- Учитывать естественную очередность операций.
- Поддерживать числа `inf`, `+inf`, `-inf`, `nan`.
- Корректно обрабатывать деление на ноль и операции с бесконечностями.

В случае возникновения посторонних символов (буквы, знаки препинания) калькулятор должен их игнорировать, продолжая заданные вычисления.

Входные данные: строка, являющаяся первым числом; символ, обозначающий арифметическое действие; строка, являющаяся вторым числом.

Выходные данные: результат выполненной операции.

3.1 Спецификация

Спецификация программы представлена в таблице 1.

Таблица 1. Спецификация

Входные данные	Результат	Логирование
3+5	8	[INFO] - Вычисление выражения: 3+5 [INFO] - Токенизация: 3+5 -> ['3', '+', '5'] [INFO] - Результат вычисления: 8.0
h-4	-4.0	[INFO] - Вычисление выражения: h-4 [INFO] - Токенизация: h-4 -> ['-', '4'] [INFO] - Результат вычисления: -4.0

Входные данные	Результат	Логирование
4/0	inf	[INFO] - Вычисление выражения: 4/0 [INFO] - Токенизация: 4/0 -> ['4', '/', '0'] [INFO] - Результат вычисления: inf
3+27/3	12.0	[INFO] - Вычисление выражения: 3+27/3 [INFO] - Токенизация: 3+27/3 -> ['3', '+', '27', '/', '3'] [INFO] - Результат вычисления: 12.0
s s	0.0	[INFO] - Вычисление выражения: s s [INFO] - Токенизация: s s -> [] [WARNING] - Не удалось разобрать выражение на токены
_	4.0	[INFO] - Вычисление выражения: _ [INFO] - Токенизация: _ -> ['_'] [INFO] - Результат вычисления: 4.0
5*-inf	-inf	[INFO] - Вычисление выражения: 5*-inf [INFO] - Токенизация: 5*-inf -> ['5', '*', '-', 'inf'] [INFO] - Результат вычисления: -inf

3.2 Исходный код программы

Листинг 1. Исходный код программы

```

1 import re
2 import math
3 from typing import List, Tuple
4 class Calculator:
5     def __init__(self):
6         self.previous_result = None
7         self.setup_logging()
8
9     def setup_logging(self):
10        import logging
11        self.logger = logging.getLogger(__name__)
12
13    def tokenize(self, expr: str) -> List[str]:
14
15    pattern = r'''
16    \d+\.\d* | # числа
17    \.\d+ | # числа начинающиеся с точки
18    [+ \- * / ^ ( ) ] | # операторы и скобки

```

```

19 _          | # предыдущий результат
20 inf        | # бесконечность
21 \+inf      | # положительная бесконечность
22 -inf       | # отрицательная бесконечность
23 nan        | # не число
24 '''
25 re.compile(pattern)
26 t = re.findall(pattern, expr, re.VERBOSE | re.IGNORECASE)
27 t = [token.strip() for token in t if token.strip()]
28 self.logger.info(f"Токенизация: {expr} -> {t}")
29 return t
30
31 def is_n(self, token: str) -> bool:
32 if token.lower() in ['inf', '+inf', '-inf', 'nan']:
33 return True
34 try:
35 float(token)
36 return True
37 except ValueError:
38 return False
39
40 def to_n(self, token: str) -> float:
41 token_lower = token.lower()
42 if token_lower == 'inf' or token_lower == '+inf':
43 return float('inf')
44 elif token_lower == '-inf':
45 return float('-inf')
46 elif token_lower == 'nan':
47 return float('nan')
48 else:
49 return float(token)
50
51 def apply_operator(self, operator: str, l: float, r: float) -> float:
52 self.logger.debug(f"Применение оператора: {l} {operator} {r}")
53 try:
54 match operator:
55 case '+':
56 result = l + r
57 case '-':
58 result = l - r
59 case '*':
60 result = l * r
61 case '/':
62 if r == 0:
63 if l == 0:
64 result = float('nan')
65 elif l > 0:
66 result = float('inf')
67 elif l < 0:
68 result = float('-inf')
69 else:
70 result = l / r
71 else:
72 result = l / r
73 case '^':
74 if l == 0 and r < 0:

```

```

75 result = float('inf')
76 else:
77 result = l ** r
78 case _:
79 raise ValueError(f"Неизвестный оператор: {operator}")
80 self.logger.debug(f"Результат операции: {result}")
81 return result
82 except Exception as e:
83 self.logger.error(f"Ошибка при применении оператора {operator}: {e}")
84 return float('nan')
85
86 def parse_expr(self, t: List[str], min_priority: int = 1) -> Tuple[float
    , List[str]]:
87 PRIORITY = {
88     '+': 1, '-': 1,
89     '*': 2, '/': 2,
90     '^': 3,
91 }
92 l, t = self.parse_primary(t)
93 while t and t[0] in PRIORITY:
94     operator = t[0]
95     if PRIORITY[operator] < min_priority:
96         break
97     next_precedence = PRIORITY[operator] + 1 if operator == '^' else
        PRIORITY[operator]
98     r, t = self.parse_expr(t[1:], next_precedence)
99     l = self.apply_operator(operator, l, r)
100 return l, t
101
102 def parse_primary(self, t: List[str]) -> Tuple[float, List[str]]:
103 if not t:
104     self.logger.error("Неожиданный конец выражения")
105     return float('nan'), []
106 token = t[0]
107 if token == '-':
108     r, remaining = self.parse_primary(t[1:])
109     return -r, remaining
110 if token == '(':
111     result, remaining = self.parse_expr(t[1:])
112     if remaining and remaining[0] == ')':
113         return result, remaining[1:]
114     self.logger.error("Незакрытая скобка")
115     return float('nan'), []
116 if token == '_':
117     if self.previous_result is None:
118         self.logger.warning("Попытка использовать предыдущий результат, но он не
            определен")
119     return 0.0, t[1:]
120 return self.previous_result, t[1:]
121 if self.is_n(token):
122     return self.to_n(token), t[1:]
123 self.logger.error(f"Неожиданный токен: {token}")
124 return float('nan'), t[1:]
125
126 def calculate(self, expr: str) -> float:
127 self.logger.info(f"Вычисление выражения: {expr}")

```

```

128 try:
129     expr = expr.replace(' ', '')
130     if not expr:
131         self.logger.warning("Пустое выражение")
132         return 0.0
133     t = self.tokenize(expr)
134     if not t:
135         self.logger.warning("Не удалось разобрать выражение на токены")
136         return 0.0
137     result, remaining_t = self.parse_expr(t)
138     if remaining_t:
139         self.logger.warning(f"Необработанные токены: {remaining_t}")
140     self.previous_result = result
141     self.logger.info(f"Результат вычисления: {result}")
142     return result
143 except Exception as e:
144     self.logger.error(f"Критическая ошибка при вычислении: {e}")
145     return float('nan')
146
147
148 def main():
149     calc = Calculator()
150     calc.logger.info("Калькулятор запущен")
151     print("Интерактивный калькулятор")
152     print("Введите 'quit' для выхода")
153     print("-" * 40)
154     while True:
155         try:
156             expr = input("> ").strip()
157             if expr.lower() in ['quit', 'exit', 'q']:
158                 calc.logger.info("Калькулятор завершил работу")
159                 break
160             if not expr:
161                 continue
162             result = calc.calculate(expr)
163             print(f"Результат: {result}")
164             print()
165         except KeyboardInterrupt:
166             calc.logger.info("Работа прервана пользователем")
167             break
168         except Exception as e:
169             calc.logger.error(f"Неожиданная ошибка: {e}")
170             print(f"Ошибка: {e}")
171
172 if __name__ == "__main__":
173     main()
174
175 import logging
176 from calculator import Calculator
177
178 def setup_logging():
179     logger = logging.getLogger()
180     logger.setLevel(logging.DEBUG)
181     file_formatter = logging.Formatter(
182         '[%(asctime)s] - [%(name)s] - [%(levelname)s] - [%(funcName)s:%(lineno)d] - [%(message)s]')

```

```

183 console_formatter = logging.Formatter('[%(levelname)s] - %(message)s')
184 file_handler = logging.FileHandler('calculator.log', encoding='UTF-8')
185 file_handler.setLevel(logging.DEBUG)
186 file_handler.setFormatter(file_formatter)
187 console_handler = logging.StreamHandler()
188 console_handler.setLevel(logging.INFO)
189 console_handler.setFormatter(console_formatter)
190 logger.addHandler(file_handler)
191 logger.addHandler(console_handler)
192
193
194 def main():
195     setup_logging()
196     calc = Calculator()
197     calc.logger.info("Калькулятор запущен")
198     while True:
199         try:
200             expression = input("> ").strip()
201             if expression.lower() in ['quit', 'exit', 'q']:
202                 calc.logger.info("Калькулятор завершил работу")
203                 break
204             if not expression:
205                 continue
206             result = calc.calculate(expression)
207             print(f"{result}")
208         except KeyboardInterrupt:
209             calc.logger.info("Работа прервана пользователем")
210             break
211         except Exception as e:
212             calc.logger.error(f"Неожиданная ошибка: {e}")
213             print(f"Ошибка: {e}")
214
215 if __name__ == "__main__":
216     main()

```

4 Описание проведенной инспекции кода

В заседании участвовали:

- Программист - разработчик программы - Мартыненко Анна;
- Специалист по тестированию - Михалец Мартин;
- Секретарь - Мелещенко Софья.

В начале заседания программистом была описана логика программы и представлена ее спецификация. В процессе инспекции были заданы вопросы с целью выявления ошибок и несоответствия спецификации. Секретарем был составлен протокол заседания с записанными вопросами и ответами на них, а также рекомендации по исправлению несоответствий. Далее представлены заданные вопросы и ответы на них.

4.1 Протокол заседания

1. На каком языке написана программа?

Ответ: на языке Python.

2. Используются ли переменные с неинициализированными значениями?

Ответ: нет.

3. Выходят ли индексы за границы массива?

Ответ: нет.

4. Используются ли нецелочисленные индексы?

Ответ: нет.

5. Есть ли “висячие” ссылки?

Ответ: нет.

6. Корректны ли атрибуты всех псевдонимов?

Ответ: да.

7. Сопласуются ли атрибуты структуры данных и физической записи?

Ответ: да.

8. Вычисляются ли адреса битовых строк? Передаются ли битовые строки в качестве аргументов?

Ответ: нет.

9. Корректно ли используются атрибуты памяти, адресуемой с помощью ссылок и указателей?
Ответ: да.
10. Согласуются ли описания структуры данных в разных процедурах?
Ответ: да.
11. Существуют ли ошибки завышения или занижения значения индекса на единицу при индексации массивов?
Ответ: нет.
12. Соблюдены ли правила наследования объектов?
Ответ: да.
13. Участвуют ли в вычислениях неарифметические переменные?
Ответ: нет.
14. Выполняются ли вычисления с использованием данных разного типа?
Ответ: да, но это особенность языка разработки - в Python поддерживается динамическая типизация.
15. Выполняются ли вычисления с переменными разной длины?
Ответ: нет.
16. Присваиваются ли переменным значения типа данных большего размера?
Ответ: нет.
17. Возможны ли переполнение или потеря точности в процессе промежуточных вычислений?
Ответ: да.
18. Возможно ли деление на нуль?
Ответ: нет.
19. Критичны ли погрешности, возникающие из-за использования двоичных представлений чисел при вычислениях?
Ответ: нет.
20. Не выходят ли значения переменных за логически обоснованные пределы?
Ответ: нет.

21. Понятны ли приоритеты операций?
Ответ: да.
22. Правильно ли используются результаты деления целых чисел?
Ответ: да.
23. Все ли переменные объявлены?
Ответ: да.
24. Понятны ли последствия использования атрибутов по умолчанию?
Ответ: да.
25. Правильно ли инициализированы массивы и строки?
Ответ: да.
26. Правильно ли определены размеры, типы и классы памяти?
Ответ: да.
27. Согласуется ли инициализация с классом памяти?
Ответ: да.
28. Имеются ли переменные с похожими именами?
Ответ: да.
29. Есть ли попытки сравнения несопоставимых переменных?
Ответ: нет.
30. Есть ли попытки сравнения переменных разного типа?
Ответ: нет.
31. Корректно ли используются операторы сравнения?
Ответ: да.
32. Корректно ли используются булевы выражения?
Ответ: да.
33. Корректно ли используются результаты сравнения в булевых выражениях?
Ответ: да.
34. Корректно ли сравниваются дробные величины?
Ответ: нет.
35. Понятны ли приоритеты операций?
Ответ: да.

36. Понятен ли способ разбора булевых выражений компилятором?
Ответ: да.
37. Может ли значение индекса в переключателе превысить число переходов?
Ответ: нет.
38. Будет ли завершен каждый цикл?
Ответ: да.
39. Будет ли завершена программа?
Ответ: да.
40. Есть ли цикл, который может не выполниться из-за входных условий?
Ответ: да.
41. Корректны ли возможные погружения в цикл?
Ответ: да.
42. Имеются ли ошибки диапазона, приводящие к отклонению количества итераций от нужного значения?
Ответ: нет.
43. Соответствует ли каждому оператору DO свой оператор END?
Ответ: да.
44. Существуют ли точки ветвления, в которых не учтены все возможные условия?
Ответ: нет.
45. Правильно ли заданы атрибуты файлов?
Ответ: да.
46. Корректны ли инструкции OPEN?
Ответ: да.
47. Согласуются ли спецификации формата с инструкциями ввода-вывода?
Ответ: да.
48. Соответствует ли размер буфера размеру записи?
Ответ: да.

49. Открываются ли файлы перед обращением к ним?
Ответ: да.
50. Закрываются ли файлы после обращения к ним?
Ответ: нет.
51. Корректно ли обрабатываются признаки конца файла?
Ответ: да.
52. Обрабатываются ли ошибки ввода вывода?
Ответ: да.
53. Присутствуют ли в выходной информации орфографические или грамматические ошибки?
Ответ: нет.
54. Совпадает ли число формальных параметров с числом аргументов?
Ответ: да.
55. Совпадают ли атрибуты параметров и аргументов?
Ответ: да.
56. Совпадают ли единицы измерения параметров и аргументов?
Ответ: да.
57. Совпадает ли число аргументов, передаваемых вызываемым модулям, с числом ожидаемых параметров?
Ответ: да.
58. Совпадают ли атрибуты аргументов, передаваемых вызываемым модулям, с атрибутами ожидаемых параметров?
Ответ: да.
59. Совпадают ли единицы измерения аргументов, передаваемых вызываемым модулям, с единицами измерения ожидаемых параметров?
Ответ: да.
60. Правильно ли заданы количество, атрибуты и порядок следования аргументов при вызове встроенных функций?
Ответ: да.
61. Имеются ли ссылки на параметры, не связанные с текущей точкой входа?
Ответ: нет.

62. Делаются ли в подпрограмме попытки изменить входные аргументы?

Ответ: нет.

63. Согласуются ли определения глобальных переменных в разных модулях?

Ответ: да.

64. Передаются ли константы в качестве аргументов?

Ответ: нет.

65. Есть ли в таблице перекрестных ссылок переменные, на которые нет ссылок?

Ответ: нет.

66. Совпадает ли список атрибутов с тем, который следовало ожидать?

Ответ: да.

67. Выдаются ли какие-нибудь предупреждения или информационные сообщения при компиляции?

Ответ: да.

68. Осуществляется ли проверка корректности входных данных?

Ответ: да.

69. Нет ли пропущенных функций?

Ответ: нет.

70. Существуют ли комментарии к реализованным функциям?

Ответ: нет.

4.2 Полученные рекомендации

В результате инспекции кода были получены следующие рекомендации:

- Добавить обработку переполнения для всех арифметических операций.
- Необходимо переименовать переменные, сделав их имена более осмысленными.
- Изменить сравнение дробных чисел с использованием допуска погрешности.

- Добавить явное закрытие файловых обработчиков.
- Добавить поясняющие комментарии к реализованным функциям.

4.3 Исправление кода программы

Переименование переменных и функций:

- $t \Rightarrow \text{tokens}$;
- $\text{is_n} \Rightarrow \text{is_number}$;
- $\text{to_n} \Rightarrow \text{to_number}$;
- $l \Rightarrow \text{left}$;
- $r \Rightarrow \text{right}$;
- $\text{expr} \Rightarrow \text{expression}$.

Листинг 2. Исправленный код программы

```

1 import re
2 import math
3 from typing import List, Tuple
4
5 class Calculator:
6     def __init__(self):
7         """Инициализация калькулятора: предыдущий результат = None, настройка
            логирования"""
8         self.previous_result = None
9         self.setup_logging()
10
11     def setup_logging(self):
12         """Настройка логгера для экземпляра калькулятора"""
13         import logging
14         self.logger = logging.getLogger(__name__)
15
16     def tokenize(self, expression: str) -> List[str]:
17         """Разбивает выражение на токены"""
18         pattern = r'''
19 \d+\.\d*      | # числа
20 \.\d+        | # числа начинающиеся с точки
21 [+\-*/^()]   | # операторы и скобки
22 -            | # предыдущий результат
23 inf          | # бесконечность
24 \+inf        | # положительная бесконечность
25 -inf         | # отрицательная бесконечность
26 nan          | # не число
27 '''
28 re.compile(pattern)
29 tokens = re.findall(pattern, expression, re.VERBOSE | re.IGNORECASE)

```

```

30 tokens = [token.strip() for token in tokens if token.strip()]
31
32 self.logger.info(f"Токенизация: {expression} -> {tokens}")
33 return tokens
34
35 def is_number(self, token: str) -> bool:
36     """Проверяет, является ли токен числом или специальным значением. """
37     if token.lower() in ['inf', '+inf', '-inf', 'nan']:
38         return True
39     try:
40         float(token)
41         return True
42     except ValueError:
43         return False
44
45 def to_number(self, token: str) -> float:
46     """Преобразует токен в число с плавающей точкой. """
47     token_lower = token.lower()
48     if token_lower == 'inf' or token_lower == '+inf':
49         return float('inf')
50     elif token_lower == '-inf':
51         return float('-inf')
52     elif token_lower == 'nan':
53         return float('nan')
54     else:
55         return float(token)
56
57 def are_floats_equal(self, a: float, b: float, epsilon: float = 1e-10)
58     -> bool:
59     """Сравнение двух дробных чисел с заданной точностью"""
60     if a == b:
61         return True
62     if math.isnan(a) and math.isnan(b):
63         return True
64     if math.isinf(a) and math.isinf(b):
65         return (a > 0) == (b > 0)
66     return abs(a - b) <= max(epsilon, epsilon * max(abs(a), abs(b)))
67
68 def apply_operator(self, operator: str, left: float, right: float) ->
69     float:
70     """Применяет оператор к двум операндам. """
71     self.logger.debug(f"Применение оператора: {left} {operator} {right}")
72     try:
73         '''ОБРАБОТКА
74         ПЕРЕПОЛНЕНИЯ
75         '''
76         with np.errstate(all='raise'): # Включаем генерацию исключений при
77             переполнении
78             match operator:
79                 case '+':
80                     result = left + right
81                     if abs(result) == float('inf'):
82                         self.logger.warning(f"Переполнение при сложении: {left} + {right}")
83                 case '-':
84                     result = left - right
85                     if abs(result) == float('inf'):

```

```

83 self.logger.warning(f"Переполнение при вычитании: {left} - {right}")
84 case '*':
85     result = left * right
86     if abs(result) == float('inf'):
87         self.logger.warning(f"Переполнение при умножении: {left} * {right}")
88 case '/':
89     if self.are_floats_equal(right, 0.0):
90     if self.are_floats_equal(left, 0.0):
91         result = float('nan')
92     elif left > 0:
93         result = float('inf')
94     elif left < 0:
95         result = float('-inf')
96     else:
97         result = left / right
98     else:
99         result = left / right
100 case '^':
101     if abs(left) > 1 and right > 100:
102     self.logger.warning(f"Возможно переполнение при возведении в степень: {left}^{
        right}")
103     if self.are_floats_equal(left, 0.0):
104     if right < 0:
105         result = float('inf')
106     elif self.are_floats_equal(right, 0.0):
107         result = 1.0
108     else:
109         result = 0.0
110     elif left < 0 and not self.are_floats_equal(right, round(right)):
111         result = float('nan')
112     else:
113         result = left ** right
114 case _:
115     raise ValueError(f"Неизвестный оператор: {operator}")
116
117 self.logger.debug(f"Результат операции: {result}")
118 return result
119
120 except Exception as e:
121     self.logger.error(f"Ошибка при применении оператора {operator}: {e}")
122     return float('nan')
123 except OverflowError:
124     self.logger.error(f"Переполнение при операции {operator}")
125     return float('inf') if left > 0 and right > 0 else float('-inf')
126 except FloatingPointError as e:
127     self.logger.error(f"Ошибка плавающей точки: {e}")
128     return float('nan')
129
130
131 def parse_expression(self, tokens: List[str], min_priority: int = 1) ->
    Tuple[float, List[str]]:
132     """Рекурсивный парсер выражений с учетом приоритета операторов. Использует алгоритм
        рекурсивного спуска для разбора инфиксных выражений."""
133     PRIORITY = {
134         '+': 1, '-': 1,
135         '*': 2, '/': 2,

```

```

136     '^': 3,
137 }
138
139 left, tokens = self.parse_primary(tokens)
140
141 while tokens and tokens[0] in PRIORITY:
142     operator = tokens[0]
143     if PRIORITY[operator] < min_priority:
144         break
145
146     next_precedence = PRIORITY[operator] + 1 if operator == '^' else
        PRIORITY[operator]
147     right, tokens = self.parse_expression(tokens[1:], next_precedence)
148     left = self.apply_operator(operator, left, right)
149
150     return left, tokens
151
152 def parse_primary(self, tokens: List[str]) -> Tuple[float, List[str]]:
153     """Парсит первичные элементы выражения: числа, унарный минус, скобки, переменную _ .
        """
154     if not tokens:
155         self.logger.error("Неожиданный конец выражения")
156         return float('nan'), []
157
158     token = tokens[0]
159     if token == '-':
160         right, remaining = self.parse_primary(tokens[1:])
161         return -right, remaining
162
163     if token == '(':
164         result, remaining = self.parse_expression(tokens[1:])
165         if remaining and remaining[0] == ')':
166             return result, remaining[1:]
167         self.logger.error("Незакрытая скобка")
168         return float('nan'), []
169
170     if token == '_':
171         if self.previous_result is None:
172             self.logger.warning("Попытка использовать предыдущий результат, но он не
                определен")
173         return 0.0, tokens[1:]
174         return self.previous_result, tokens[1:]
175
176     if self.is_number(token):
177         return self.to_number(token), tokens[1:]
178
179     self.logger.error(f"Неожиданный токен: {token}")
180     return float('nan'), tokens[1:]
181
182
183 def calculate(self, expression: str) -> float:
184     """Основной метод вычисления выражения. """
185     self.logger.info(f"Вычисление выражения: {expression}")
186
187     try:
188         expression = expression.replace(' ', '')

```



```

189
190 if not expression:
191     self.logger.warning("Пустое выражение")
192     return 0.0
193
194 tokens = self.tokenize(expression)
195
196 if not tokens:
197     self.logger.warning("Не удалось разобрать выражение на токены")
198     return 0.0
199
200 result, remaining_tokens = self.parse_expression(tokens)
201
202 if remaining_tokens:
203     self.logger.warning(f"Необработанные токены: {remaining_tokens}")
204
205 self.previous_result = result
206
207 self.logger.info(f"Результат вычисления: {result}")
208 return result
209
210 except Exception as e:
211     self.logger.error(f"Критическая ошибка при вычислении: {e}")
212     return float('nan')
213
214
215
216
217 def main():
218     calc = Calculator()
219     calc.logger.info("Калькулятор запущен")
220
221     print("Интерактивный калькулятор")
222     print("Введите 'quit' для выхода")
223     print("-" * 40)
224
225     while True:
226         try:
227             expression = input("> ").strip()
228
229             if expression.lower() in ['quit', 'exit', 'q']:
230                 calc.logger.info("Калькулятор завершил работу")
231                 break
232
233             if not expression:
234                 continue
235
236             result = calc.calculate(expression)
237             print(f"Результат: {result}")
238             print()
239
240         except KeyboardInterrupt:
241             calc.logger.info("Работа прервана пользователем")
242             break
243         except Exception as e:
244             calc.logger.error(f"Неожиданная ошибка: {e}")

```

```

245 print(f"Ошибка: {e}")
246
247 if __name__ == "__main__":
248     main()
249
250 import logging
251 from calculator import Calculator
252
253
254 def setup_logging():
255     """Настройка системы логирования."""
256     logger = logging.getLogger()
257     logger.setLevel(logging.DEBUG)
258
259     file_formatter = logging.Formatter(
260         '%(asctime)s] - %(name)s] - %(levelname)s] - %(funcName)s:%(lineno)d
            ] - %(message)s]')
261
262     console_formatter = logging.Formatter('%(levelname)s] - %(message)s]')
263     '''ЯВНОЕ
264     ЗАКРЫТИЕ ФАЙЛОВЫХ ДЕСКРИПТОРОВ
265     '''
266     for handler in logger.handlers[:]:
267         logger.removeHandler(handler)
268         handler.close()
269
270     file_handler = logging.FileHandler('calculator.log', encoding='UTF-8')
271     file_handler.setLevel(logging.DEBUG)
272     file_handler.setFormatter(file_formatter)
273
274     console_handler = logging.StreamHandler()
275     console_handler.setLevel(logging.INFO)
276     console_handler.setFormatter(console_formatter)
277
278     logger.addHandler(file_handler)
279     logger.addHandler(console_handler)
280
281     return file_handler, console_handler
282
283
284 def main():
285     setup_logging()
286
287     calc = Calculator()
288     calc.logger.info("Калькулятор запущен")
289     while True:
290         try:
291             expression = input("> ").strip()
292
293             if expression.lower() in ['quit', 'exit', 'q']:
294                 calc.logger.info("Калькулятор завершил работу")
295                 break
296
297             if not expression:
298                 continue
299

```

```
300 result = calc.calculate(expression)
301 print(f"{result}")
302
303 except KeyboardInterrupt:
304     calc.logger.info("Работа прервана пользователем")
305     break
306 except Exception as e:
307     calc.logger.error(f"Неожиданная ошибка: {e}")
308     print(f"Ошибка: {e}")
309
310 if __name__ == "__main__":
311     main()
```

Заключение

В ходе выполнения данной лабораторной работы был изучен метод ручного тестирования - инспекция кода. Было проведено инспекционное заседание, в котором участвовали секретарь, программист и специалист по тестированию.

В результате инспекции был обнаружен ряд недочетов и ошибок, а также сформулированы рекомендации для программиста по их устранению. После получения рекомендаций были внесены необходимые исправления в код. Были добавлены поясняющие комментарии, обработка переполнения, явное закрытие файловых обработчиков, а также изменена логика сравнения дробных чисел и переименованы переменные и функции для повышения их осмысленности.

В результате выполненной работы я не только исправила код исходной программы, но и получила практический опыт в проведении инспекции кода, а также улучшила понимание и производительность программы.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.